

Specyfikacja Projektu Zaliczeniowego (Programowanie w Języku Java)

Kamil Drzymała, Kamil Kasperek

13 kwietnia 2026

1. Tytuł Projektu

Przedstaw nazwę projektu, która powinna być jednoznaczna, krótka i oddawać główną ideę aplikacji.

SafeChat: Aplikacja komunikacyjna w czasie rzeczywistym z szyfrowaniem End-to-End (E2E Encryption)

2. Opis i Cele Projektu

Określ, jaki problem rozwiązuje projekt, do czego służy oraz jakie są jego główne cele funkcjonalne.

Projekt ma na celu stworzenie aplikacji typu czat dla wielu użytkowników w języku Java. Głównym problemem, który aplikacja rozwiązuje, jest brak prywatności w standardowych komunikatorach – w proponowanym systemie serwer pełni jedynie rolę przekaźnika i nie jest w stanie odczytać treści wiadomości. Główne cele projektu to:

- Implementacja stabilnej komunikacji w czasie rzeczywistym w oparciu o gniazda sieciowe (Java Sockets).
- Zapewnienie pełnej poufności przesyłanych wiadomości z wykorzystaniem kryptografii asymetrycznej (RSA) oraz symetrycznej (AES).
- Zbudowanie prostego i intuicyjnego interfejsu graficznego (GUI) dla użytkownika końcowego.
- Równoległa obsługa wielu klientów za pomocą wielowątkowości po stronie serwera.

3. Wymagania Funkcjonalne

Lista konkretnych funkcji, które aplikacja powinna oferować użytkownikowi końcowemu.

- Nawiązywanie i zrywanie połączenia z serwerem komunikacyjnym.

- Wybór unikalnego pseudonimu (nicku) przy logowaniu.
- Automatyczne generowanie par kluczy kryptograficznych po stronie klienta.
- Wymiana kluczy publicznych między klientami za pośrednictwem serwera.
- Wysyłanie i odbieranie zaszyfrowanych wiadomości w oknach czatu prywatnego.

4. Wymagania Niefunkcjonalne

Wymagania techniczne, jakościowe i pozafunkcjonalne, które aplikacja powinna spełniać.

- **Wydajność:** Płynna obsługa minimum 50 równocześnie podłączonych użytkowników na jednym węźle serwera.
- **Bezpieczeństwo:** Szyfrowanie E2E, klucze prywatne nigdy nie opuszczają lokalnego środowiska użytkownika.
- **Niezawodność:** Serwer nie zawiesza się w przypadku nagłego zerwania połączenia przez jednego z klientów.
- **Multiplatformowość:** Aplikacja uruchamia się na dowolnym systemie z zainstalowanym środowiskiem JRE (Windows, Linux, macOS).
- **Użyteczność:** Przejrzysty interfejs okienkowy zbudowany w Swing/JavaFX.

5. Architektura Systemu

Opis architektury systemu, jego głównych komponentów oraz zależności między nimi.

Typ architektury: **Klient-Serwer z elementami przesyłu P2P (logicznego).**

Schemat:

- **Serwer (Backend):** Lekka aplikacja konsolowa w Javie (ServerSocket) zarządzająca pulą wątków i rozsyłająca zaszyfrowane pakiety danych.
- **Klient (Frontend & Logika):** Aplikacja desktopowa (JavaFX/Swing) z lokalnym modułem kryptograficznym.
- **Komunikacja:** Gniazda sieciowe TCP (Java Sockets). Przesyłanie zserializowanych obiektów lub komunikatów w formacie JSON.

6. Zastosowane Algorytmy

Opis algorytmów i struktur danych wykorzystanych w aplikacji do realizacji funkcjonalności.

- **Algorytm RSA (2048 bit):** Szyfrowanie asymetryczne wykorzystane do bezpiecznej wymiany kluczy sesyjnych między dwoma klientami.
- **Algorytm AES (256 bit):** Szyfrowanie symetryczne wykorzystane do kodowania właściwej treści wiadomości, ze względu na znacznie wyższą wydajność przy dłuższych tekstach.
- **Zarządzanie wątkami:** Zastosowanie wzorca Thread Pool na serwerze do optymalnego zarządzania połączeniami.
- **Algorytmy autoryzacji sesji:** Weryfikacja unikalności pseudonimów za pomocą struktur takich jak `ConcurrentHashMap` zapewniających bezpieczeństwo wielowątkowe.

7. Struktura Klas

Opisuje strukturę klas, ich odpowiedzialności oraz podział logiczny na warstwy w aplikacji.

Współdzielone

- `MessageDTO` – obiekt transferu danych (nadawca, odbiorca, zaszyfrowana wiadomość, typ komunikatu).
- `User` – model opisujący podłączonego użytkownika.

Serwer

- `ServerMain` – inicjalizacja serwera i pętla nasłuchująca.
- `ClientHandler` – wątek obsługujący pojedyncze połączenie TCP.
- `ConnectionManager` – zarządzanie aktywnymi połączeniami.

Klient

- `ChatController` – zarządzanie zdarzeniami interfejsu użytkownika (UI).
- `NetworkService` – komunikacja z gniazdem serwera, odbieranie i wysyłanie `MessageDTO`.
- `CryptoService` – generowanie kluczy, szyfrowanie/deszyfrowanie RSA i AES.

8. Wykorzystane Technologie i Biblioteki

Opis technologii oraz narzędzi użytych w projekcie.

Język i Środowisko:

Java (wersja 17+), JVM

Sieć i Protokoły:

Wbudowane pakiety `java.net.*` oraz `java.io.*`

Kryptografia:

Java Cryptography Architecture (JCA) - pakiety `java.security.*` oraz `javax.crypto.*`

Interfejs Użytkownika:

JavaFX lub Java Swing

9. API i Modele Danych

Przykładowe punkty końcowe API oraz struktura danych wymienianych między systemami.

Ponieważ komunikacja nie opiera się na klasycznym REST API, opisano strukturę wymienianych obiektów przesyłanych przez strumień TCP.

Struktura Obiektu MessageDTO (JSON / Serializacja):

```
{
  "type": "ENUM(CHAT, KEY_EXCHANGE, JOIN, LEAVE)",
  "sender": "string (pseudonim)",
  "recipient": "string (pseudonim lub 'ALL')",
  "encryptedPayload": "byte[] (zaszyfrowana treść AES)",
  "timestamp": "long",
  "publicKey": "byte[] (opcjonalnie, tylko dla KEY_EXCHANGE)"
}
```

10. Interfejs Użytkownika

Opis interfejsu graficznego aplikacji, jego struktury, logiki działania oraz dostępnych ekranów.

Główne widoki:

- **Ekran połączenia:** Pole do wpisania adresu IP serwera, numeru portu oraz pożądanego pseudonimu (nicku) i przycisk "Połącz".
- **Główne okno komunikatora:** Podzielone na strefy:
 - *Panel boczny:* Lista aktywnych użytkowników kliknięcie w nick otwiera osobną kartę czatu prywatnego.
 - *Panel centralny:* Historia konwersacji (z podziałem na nadawcę/odbiorcę).
 - *Panel dolny:* Pole tekstowe do wprowadzania wiadomości oraz przycisk "Wyślij".

Elementy UX/UI:

- Automatyczne scrollowanie historii wiadomości do najnowszej.
- Kolorystyczne rozróżnienie wiadomości własnych od cudzych.
- Obsługa skrótu klawiszowego *Enter* do wysyłania wiadomości.

11. Etapy Realizacji / Harmonogram

Plan wdrożenia projektu z podziałem na fazy i szacowanym czasem realizacji.

Faza	Opis	Czas
Planowanie	Projektowanie protokołu komunikacji i struktury obiektów	1 tydzień
Backend / Sieć	Implementacja serwera i połączeń Sockets na kliencie	2 tygodnie
Kryptografia	Integracja AES/RSA, bezpieczna wymiana kluczy	2 tygodnie
Interfejs (GUI)	Budowa widoków (JavaFX/Swing) i podpięcie logiki	2 tygodnie
Testowanie	Testy wielowątkowe, weryfikacja poprawności szyfrowania	1 tydzień

12. Testowanie

Sposoby testowania aplikacji, typy testów oraz używane narzędzia.

- **Testy jednostkowe (JUnit):** Sprawdzenie poprawności algorytmów szyfrujących (czy proces *encrypt* → *decrypt* zwraca oryginał).
- **Testy integracyjne:** Weryfikacja przesyłu obiektów zserializowanych na linii klient-serwer.
- **Testy manualne (E2E):** Uruchomienie serwera oraz kilku instancji klienta lokalnie celem sprawdzenia płynności czatu i obsługi wyjątków.

13. Ryzyka i Środki Zaradcze

Identyfikacja potencjalnych zagrożeń projektowych i strategii ich ograniczenia.

Ryzyko	Wpływ	Zarządzanie
Zastój w wątkach	Wysoki	Dokładne zaplanowanie synchronizacji, użycie kolekcji wielowątkowych

Wycieki pamięci na serwerze	Wysoki	Właściwe zamykanie strumieni (I/O)
Skomplikowana logika kluczy (E2E)	Średni	Etapowe wdrożenie: najpierw działający czat bez szyfrowania, potem dodanie warstwy Crypto

14. Kryteria Sukcesu

Opisz po czym poznać, że projekt zakończył się sukcesem.

- Serwer stabilnie utrzymuje połączenia i natychmiastowo przekazuje wiadomości.
- Wiadomości nie da się odczytać podczas sniffingu ruchu sieciowego (np. Wireshark), a po stronie serwera figurują one jako ciąg bajtów.
- Poprawnie działający proces wymiany kluczy publicznych.
- Kod aplikacji nie wyrzuca nieobsłużonych wyjątków w konsoli.

15. Szacunkowe Nakłady Pracy (Workload Estimation)

Osobne oszacowanie czasu pracy dla poszczególnych modułów i funkcji.

Całkowity szacowany czas pracy: **70-90 godzin roboczych**.

Moduł / Zadanie	Opis	Szac. czas (h)
Konfiguracja i architektura	Projekt protokołu, szkielet klas	10
Obsługa sieci (Serwer)	ServerSocket, pula wątków, broadcast	15
Obsługa sieci (Klient)	Nasłuchiwanie strumieni, łączenie	10
Moduł kryptograficzny (Crypto)	RSA/AES z użyciem <code>javax.crypto</code>	20
Interfejs użytkownika (GUI)	Ekran logowania, okno czatu, wątki UI	15
Testowanie i debugowanie	Poprawki, testy połączeń lokalnych	15
Dokumentacja kodu	Komentarze JavaDoc, czyszczenie kodu	5